

EXAMEN — MODULE DEVOPS

Mini-projet : développement d'une application avec CI/CD

Module	DevOps
Type d'évaluation	Mini-projet (3 à 4 semaines)
Modalité de travail	Binôme (2 étudiants)
Modalité d'évaluation	Dépôt Git + rapport écrit
Plateforme de dépôt	Google Classroom
Délai de dépôt	4 semaines à compter de la mise en ligne du sujet
Enseignant	Pr. Soufiane HAMIDA

1. Contexte et objectifs

Ce mini-projet vise à mettre en pratique l'ensemble des notions abordées dans le module DevOps. Chaque binôme doit développer une application logicielle et la mettre en place dans une chaîne d'intégration et de déploiement continu (CI/CD). L'évaluation porte autant sur le code applicatif que sur la qualité de la chaîne DevOps mise en œuvre autour de ce code.

À l'issue de ce projet, le binôme doit être capable de :

- Versionner un projet logiciel avec Git en suivant un workflow professionnel (branches, pull requests, revues).
- Conteneuriser une application avec Docker et la rendre exécutable de manière reproductible.
- Mettre en place un pipeline d'intégration continue qui automatise les tests et la construction des images Docker.
- Documenter le projet, son architecture et son déploiement de manière claire et professionnelle.

2. Modalités de travail

- **Travail en binôme** : le projet est obligatoirement réalisé à deux étudiants. Les binômes doivent être déclarés à l'enseignant dans la première semaine. Aucun changement de binôme n'est autorisé après cette date.
- **Durée** : 4 semaines à compter de la mise en ligne du sujet sur Google Classroom.

- **Plateforme Git/CI** : au choix du binôme (GitHub + GitHub Actions, GitLab + GitLab CI, Bitbucket + Pipelines, ou tout autre outil équivalent — Jenkins, CircleCI, Drone, etc.). Le choix doit être justifié dans le rapport.
- **Répartition du travail** : le binôme s'organise librement, mais chaque étudiant doit avoir une contribution visible dans l'historique Git (commits réguliers signés de son nom). Une répartition manifestement déséquilibrée est sanctionnée.
- **Originalité** : toute reprise de code externe doit être citée. Le plagiat entre binômes entraîne l'annulation du projet pour les groupes concernés.

3. Sujets proposés

Chaque binôme choisit un sujet parmi la liste suivante. Le choix se fait sur principe de premier arrivé, premier servi : un même sujet ne peut être pris que par deux binômes au maximum dans la promotion. Le binôme doit communiquer son choix à l'enseignant dans la première semaine.

N°	Titre	Description
01	Plateforme de gestion d'étudiants	Application web permettant l'inscription, la consultation et la mise à jour des fiches étudiants (CRUD), avec gestion des notes et export des relevés. Possibilité d'ajouter une authentification par rôle (administrateur / enseignant / étudiant).
02	API de gestion de bibliothèque	API REST de gestion d'une bibliothèque : livres, auteurs, emprunts, rendus. Recherche par titre, auteur ou catégorie. L'application doit gérer les emprunts en cours et l'historique des transactions.
03	Tableau de bord météo en temps réel	Application qui interroge une API météo publique (OpenWeather, WeatherAPI, etc.), affiche les conditions actuelles pour plusieurs villes, et conserve un historique des relevés dans une base de données pour des graphiques sur 7 jours.
04	Application de prise de notes en ligne	Application web type Markdown notes : création, édition, suppression de notes avec tags, recherche plein texte, partage par lien. L'interface doit prévisualiser le rendu Markdown en temps réel.
05	Plateforme de tickets de support (mini Helpdesk)	Système de gestion de tickets : un utilisateur ouvre un ticket avec une description et une priorité, un agent y répond, le statut évolue (Ouvert / En cours / Résolu / Fermé). Filtres et statistiques par agent.
06	Application de gestion de tâches collaboratives (Kanban)	Tableau Kanban à plusieurs colonnes (À faire / En cours / Terminé), création de cartes avec titre, description, échéance, étiquettes. Drag-and-drop côté frontend, persistance des positions côté backend.

N°	Titre	Description
07	Site e-commerce simplifié	Catalogue de produits, panier, simulation de commande (sans paiement réel). Authentification utilisateur, espace administrateur pour ajouter / modifier des produits. Stock géré en base de données.
08	Application de réservation (salle / restaurant / coiffure)	Système de réservation par créneau horaire. Le client choisit un service, un créneau disponible, et reçoit une confirmation. L'administrateur visualise le planning et peut bloquer / déplacer des créneaux.
09	Plateforme de quiz en ligne	Création de quiz par un enseignant, passage par les étudiants, calcul automatique du score, classement (leaderboard). QCM, questions à réponse libre, gestion du temps imparti.
10	Application de suivi de dépenses personnelles	L'utilisateur saisit ses dépenses et revenus avec catégorie (alimentation, transport, loisirs...), visualise des graphiques mensuels (camembert et histogramme), exporte les données en CSV / PDF.

Note : si un binôme souhaite proposer un sujet hors liste, il doit présenter sa proposition à l'enseignant avant la fin de la première semaine. Le sujet sera accepté ou non selon sa pertinence et sa charge de travail.

4. Exigences techniques

4.1. Application

- Choix libre du langage et du framework : Node.js, Python, Java, PHP, Go, etc.
- L'application doit avoir une interface utilisateur ou une API REST documentée.
- Les données doivent persister dans une base de données (relationnelle ou NoSQL).
- Au minimum un fichier README clair, un fichier de configuration externe, et un fichier décrivant les dépendances (package.json, requirements.txt, pom.xml...).
- Au moins quelques tests automatisés (unitaires ou d'intégration), exécutables en ligne de commande.

4.2. Git — gestion de versions

- Dépôt Git public ou privé (avec accès donné à l'enseignant) sur la plateforme choisie par le binôme.
- Workflow par branches : au minimum une branche main (production) et une branche develop (intégration), avec des branches de fonctionnalités (feature/*).
- Pull requests / Merge requests pour intégrer les fonctionnalités, avec relecture par l'autre membre du binôme.

- Historique de commits propre, lisible, avec messages explicites (type Conventional Commits recommandé).
- Fichier .gitignore adapté au langage utilisé.

4.3. Docker — conteneurisation

- Un Dockerfile écrit pour l'application, optimisé (multi-stage build apprécié).
- Un fichier docker-compose.yml décrivant la stack complète (application + base de données + éventuels services annexes).
- L'application doit être lançable en local en une seule commande : `docker compose up`.
- L'image doit être publiée sur un registre (Docker Hub, GitHub Container Registry, GitLab Registry...).

4.4. CI/CD — Intégration et déploiement continu

- Pipeline d'intégration continue déclenché à chaque push sur la branche principale et à chaque pull/merge request.
- Le pipeline doit au minimum : installer les dépendances, exécuter le linter, lancer les tests automatiques, construire l'image Docker.
- Publication automatique de l'image Docker sur le registre quand le pipeline réussit sur la branche main.
- Le statut du pipeline (badge de build) doit apparaître dans le README du projet.

4.5. Kubernetes (optionnel — bonus)

Cette partie est optionnelle. Les binômes qui souhaitent aller plus loin peuvent ajouter un déploiement Kubernetes.

- Manifestes Kubernetes (Deployment, Service, ConfigMap, Secret) dans un dossier k8s/ du dépôt.
- Déploiement testé sur Minikube ou un cluster équivalent.
- Captures d'écran ou logs prouvant le bon fonctionnement à inclure dans le rapport.

5. Rapport écrit

Le rapport est un document PDF de 10 à 20 pages (hors annexes), rédigé en français, déposé en même temps que le lien vers le dépôt Git.

Plan attendu

- Page de garde : nom de l'établissement, intitulé du module, titre du projet, noms des étudiants, année universitaire.
- Introduction : présentation du sujet choisi et des objectifs.
- Architecture de l'application : schéma global, choix techniques, justifications.
- Mise en œuvre : structure du code, modèle de données, principales fonctionnalités.
- Stratégie Git : workflow adopté, structure des branches, gestion des conflits.
- Conteneurisation : explication du Dockerfile et du docker-compose, captures d'écran de l'exécution.
- Pipeline CI/CD : description des étapes, captures du pipeline qui passe au vert, lien vers les exécutions.
- Kubernetes (si applicable) : manifestes, captures de déploiement.
- Difficultés rencontrées et solutions apportées.
- Répartition du travail entre les deux étudiants.
- Conclusion et perspectives.
- Annexes : extraits de code remarquables, références.

6. Livrables et procédure de dépôt

Livrables attendus

- **Le rapport au format PDF**, nommé : *Rapport_DevOps_NomEtudiant1_NomEtudiant2.pdf*
- **Un lien vers le dépôt Git** (public ou privé avec accès donné à l'enseignant).
- **Un lien vers l'image Docker publiée** sur le registre choisi.
- **Un lien vers une exécution réussie du pipeline CI/CD.**

Modalité de dépôt

Tout est déposé sur Google Classroom dans la rubrique dédiée à l'examen DevOps. Aucun envoi par e-mail ni dépôt hors plateforme ne sera accepté.

Date limite

Le dépôt doit intervenir au plus tard 4 semaines à compter de la mise en ligne du sujet sur Google Classroom (voir la date exacte affichée sur Classroom). Tout retard entraîne une pénalité décidée par l'enseignant ou une note nulle au-delà d'un certain délai.

7. Critères d'évaluation

L'évaluation porte sur les axes suivants, par ordre de priorité décroissant :

Axe	Ce qui est attendu
Qualité de la chaîne CI/CD	Pipeline correctement configuré, étapes pertinentes, pipeline qui passe au vert, publication automatique de l'image.
Maîtrise de Git	Workflow par branches, qualité des commits, utilisation des pull/merge requests, historique propre et lisible.
Conteneurisation Docker	Qualité du Dockerfile, optimisations, docker-compose fonctionnel, image publiée sur un registre.
Application	Fonctionnalités opérationnelles, qualité du code, présence de tests, persistance correcte des données.
Rapport et documentation	Clarté, structure, qualité rédactionnelle, schémas pertinents, présence d'un README complet sur le dépôt.
Bonus Kubernetes	Manifestes K8s fonctionnels, démonstration de déploiement, traces dans le rapport.

8. Règles particulières

- **Plagiat et copie** : toute copie entre binômes ou copie d'un projet existant en ligne entraîne l'annulation pure et simple du projet pour tous les groupes concernés.
- **Utilisation de l'IA générative** : autorisée comme aide à la recherche et à la rédaction. Toutefois, le code et le rapport doivent être compris par les étudiants : un binôme dont l'un des membres ne peut pas expliquer son propre code peut voir sa note revue à la baisse.
- **Dépôt Git privé** : si le binôme choisit un dépôt privé, l'enseignant doit y avoir un accès en lecture pendant toute la durée de l'évaluation.
- **Pipeline cassé au moment du dépôt** : un pipeline rouge au moment de la remise est considéré comme un projet incomplet. Vérifiez que la dernière exécution est verte avant le dépôt.
- **Questions** : toute question liée au sujet doit être posée via Google Classroom (rubrique commentaires) afin que l'ensemble de la promotion bénéficie de la réponse.